```
async Task TerminalMiddleware(HttpContext context)
{
    await context.Response.WriteAsync("That's all, folks!");
}
```

This particular middleware writes a message to the response. Control is not passed on to any other middleware components, so this type of middleware is known as *terminal middleware*. It terminates further processing within the pipeline. Terminal middleware is registered via the `WebApplication` object's `Run` method:

```
app.Run(TerminalMiddleware);
```

The `RequestDelegate` is a standard .NET delegate, so it can also be written inline as an anonymous function using a lambda expression, rather than as a named method.

Listing 2.9 Delegating bodies specified inline using a lambda expression

```
app.Run(async context =>
     await context.Response.WriteAsync("That's all, folks!")
);
```

Try registering this middleware using either approach by placing the app.Run call at the beginning of the pipeline—before the condition that checks to see if the current environment is `Development`.

Listing 2.10 Adding the middleware to the start of the pipeline

```
app.Run(async context =>
     await context.Response.WriteAsync("That's all, folks!")
);
if (app.Environment.IsDevelopment())
{
   ...
```

Then run the application. You should see the output shown in figure 2.14.